

MDC-112 Web：將 MDC 與網頁框架整合

來源：<https://codelabs.developers.google.com/codelabs/mdc-112-web?hl=zh-tw>

步驟數：6

瞭解如何將預先建立的 Material Design 元件擴充為適合任何網路架構的元件。

目錄

1. [1. 簡介](#)
2. [2. 設定開發環境](#)
3. [3. 元件組成](#)
4. [4. 實作轉接程式方法](#)
5. [5. 實作元件方法](#)
6. [6. 總結](#)

1. 簡介



開發人員可透過 Material 元件 (MDC) 實作 Material Design。MDC 由 Google 的工程師和 UX 設計師團隊打造，提供數十種美觀實用的 UI 元件，適用於 Android、iOS、網頁和 Flutter。material.io/develop

MDC Web 的設計宗旨是整合至任何前端架構，同時遵守 Material Design 原則。本程式碼研究室會引導您建構以 MDC Web 為基礎的 React 元件。本程式碼研究室中學到的原則可套用至任何 JavaScript 架構。

MDC Web 的建構方式

MDC Web 的 JavaScript 層包含每個元件的三個類別：*Component*、*Foundation* 和 *Adapter*。這個模式可讓 MDC Web 彈性地與前端架構整合。

基礎包含實作 Material Design 的商業邏輯。Foundation 不會參照任何 HTML 元素。這樣我們就能將 HTML 互動邏輯抽象化為 *Adapter*。**基礎**具有轉接器。

Adapter 是介面。**基礎**會參照 Adapter 介面，實作 Material Design 商業邏輯。您可以在 Angular 或 React 等不同架構中實作 *Adapter*。轉接程式的實作會與 DOM 結構互動。

元件具有「基礎」，其角色是

1. 使用非架構 JavaScript 實作 *Adapter*，以及
2. 提供可代理 *Foundation* 中方法的方法。

MDC Web 提供哪些功能

MDC Web 中的每個套件都隨附元件、基礎和介面卡。如要例項化 *Component*，您必須將根 [element](#) 傳遞至 *Component* 的建構函式方法。元件會實作 *Adapter*，與 DOM 和 HTML 元素互動。接著，*Component* 會例項化 *Foundation*，後者會呼叫 *Adapter* 方法。

如要將 MDC Web 整合至框架，您必須以該框架的語言/語法建立自己的「元件」。架構 *Component* 會實作 MDC Web 的 *Adapter*，並使用 MDC Web 的 *Foundation*。

建構項目

本程式碼研究室會示範如何建構自訂 *Adapter*，使用 *Foundation* 邏輯實作 Material Design React 元件。涵蓋「[整合至架構](#)」中的進階主題。本程式碼研究室使用 React 做為範例架構，但這種做法適用於任何其他架構。

在本程式碼研究室中，您將建構頂端應用程式列，並重新建立頂端應用程式列的[示範頁面](#)。示範頁面版面配置已設定完成，因此您可以開始處理頂端應用程式列。頂端應用程式列會包含：

- 導覽圖示
- 待辦事項
- 共有 4 種變體：簡短、一律摺疊、固定和顯眼變體

事前準備：

- 最新版 [Node.js](#) (隨附 JavaScript 套件管理工具 [npm](#))
- 程式碼範例 (將在下一個步驟中下載)

- 具備 HTML、CSS、JavaScript 和 React 的基本知識
-

步驟 2 · 約 5 分鐘

2. 設定開發環境

下載程式碼研究室的入門應用程式

範例應用程式位於 `material-components-web-codelabs-master/mdc-112/starter` 目錄中。

請注意根目錄名稱中的 `-master` 後置字元。如果您是從上方按鈕下載範例應用程式，請在參照根目錄時，一律附加 `-master` 後置字元，以用於本程式碼研究室的其餘部分。

...或從 GitHub 複製

如要從 GitHub 複製本程式碼研究室，請執行下列指令：

```
git clone https://github.com/material-components/material-components-web-codelabs
cd material-components-web-codelabs/mdc-112/starter
```

如需更多說明，請參閱「[從 GitHub 複製存放區](#)」。

安裝專案依附元件

從入門目錄 `material-components-web-codelabs/mdc-112/starter` 執行：

```
npm install
```

您會看到許多活動，最後終端機應該會顯示安裝成功：

```
added 1017 packages from 692 contributors in 13.974s
```

MDC Web 使用名為 [Sass](#) 的 CSS 前置處理器。如果您使用其他 CSS 前置處理器 (或完全不使用)，也沒問題！您可以[安裝 sass CLI](#)，或在建構工具鍊中使用外掛程式 (例如 Webpack 的 [sass-loader](#)、Gulp 的 [gulp-sass](#) 或 Grunt 的 [grunt-sass](#))，在目前使用的前置處理器旁使用 Sass。

為了讓程式碼在 IE 11 等舊版瀏覽器中運作，我們使用名為 [Babel](#) 的轉譯器，將 MDC Web 的 [ES2015](#) JavaScript 語法轉譯為 ES5 JavaScript 語法。

我們會使用 [webpack](#) 組合檔案，並在本機提供網站。這項專案的 webpack 設定已在 `webpack.config.js` 檔案中設定完成。如要進一步瞭解設定方式，請參閱[入門指南](#)。

執行範例應用程式

在相同目錄中執行：

```
npm start
```

`webpack-dev-server` 就會開始。將瀏覽器設為指向 <http://localhost:8080/>，即可查看網頁。

```
mattgoo@mattgoo-macbookpro:~/Documents/material-components-web-codelabs/mdc-112/starter (master) $ npm start
> mdc-112-web@0.0.0 start /Users/mattgoo/Documents/material-components-web-codelabs/mdc-112/starter
> webpack-dev-server

Project is running at http://localhost:8080/
webpack output is served from /
```

如果無法順利執行 `npm install` 或 `npm start`，請停止並排解開發人員環境問題。

太棒了，瀏覽器應會執行 Top App Bar React 示範頁面的範例程式碼。您應該會看到一連串的 [lorem ipsum](#) 文字、右下方的「Controls」方塊，以及未完成的頂端應用程式列：

TOP APP BAR

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

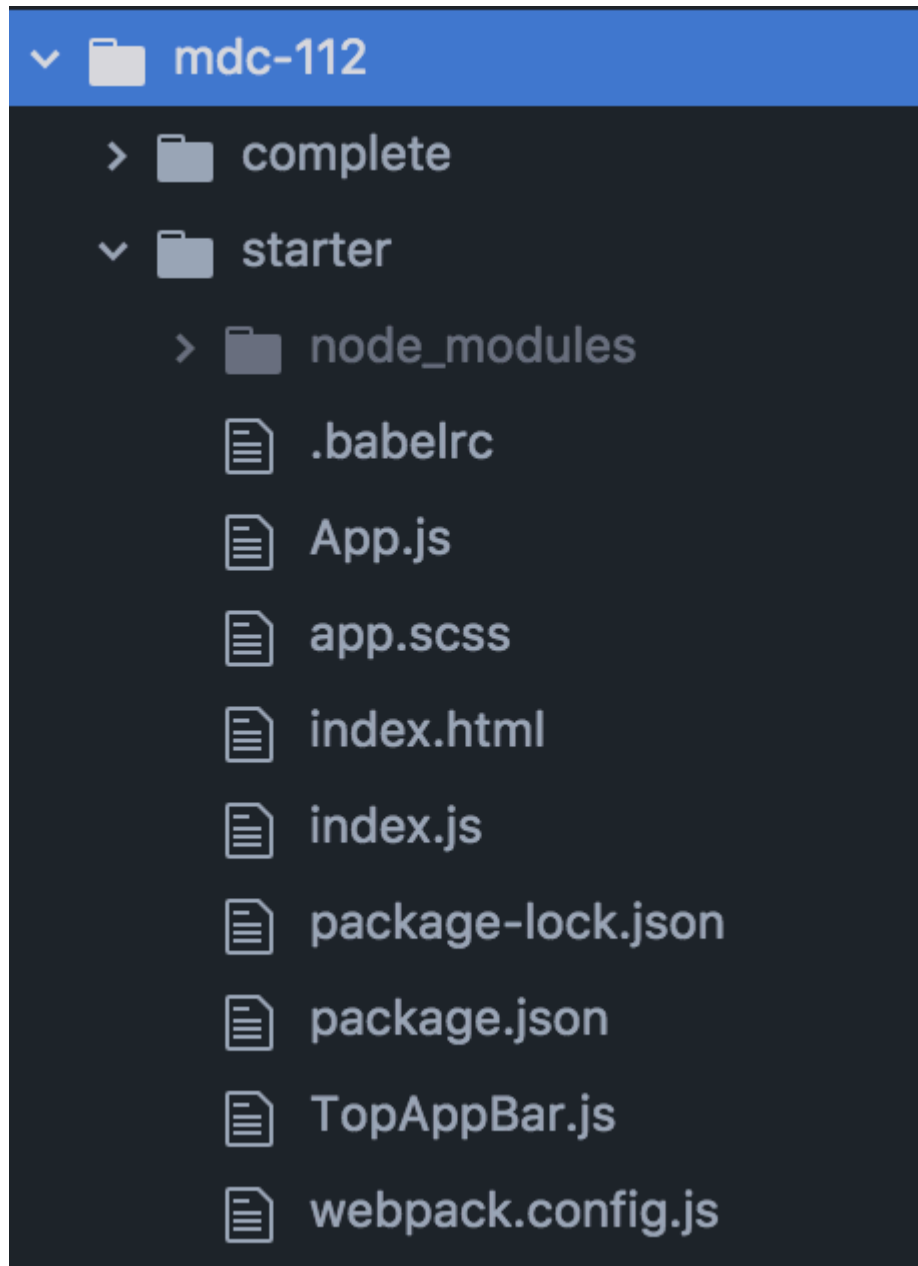
Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est.

Controls

- Short ☐
- Always Short ☐
- RTL ☐
- Prominent ☐
- No Action Items ☐

查看程式碼和專案

開啟程式碼編輯器後，專案目錄應如下所示：



開啟 `App.js` 檔案，然後查看 `render` 方法，其中包含 `<TopAppBar>` 元件：

App.js

```

render() {
  const {isFixed, isShort, isRtl, isProminent, isAlwaysCollapsed, shouldReinit} =
this.state;

  return (
    <section
      dir={isRtl ? 'rtl' : 'ltr'}
      className='mdc-typography'>
      {
        shouldReinit ? null :
        <TopAppBar
          navIcon={this.renderNavIcon()}
          short={isShort}
          prominent={isProminent}
          fixed={isFixed}
          alwaysCollapsed={isAlwaysCollapsed}
          title='Mountain View, CA'
          actionItems={this.actionItems}
        />
      }
      <div className={classnames('mdc-top-app-bar--fixed-adjust', {
        'mdc-top-app-bar--short-fixed-adjust': isShort || isAlwaysCollapsed,
        'mdc-top-app-bar--prominent-fixed-adjust': isProminent,
      })}>
        {this.renderDemoParagraphs()}
      </div>

      {this.renderControls()}
    </section>
  );
}

```

這是應用程式中 `TopAppBar` 的進入點。

開啟 `TopAppBar.js` 檔案，這是具有 `render` 方法的裸露 React `Component` 類別：

TopAppBar.js

```

import React from 'react';

export default class TopAppBar extends React.Component {
  render() {
    return (
      <header>
        TOP APP BAR
      </header>
    );
  }
}

```


3. 元件組成

在 React 中，`render` 方法會輸出元件的 HTML。頂端應用程式列元件會算繪 `<header />` 標記，並由 2 個主要區段組成：

1. 導覽圖示和標題部分
2. 動作圖示部分

如要瞭解構成頂端應用程式列的元素，請參閱 [GitHub](#) 上的說明文件。

將 `TopAppBar.js` 中的 `render()` 方法修改成如下所示：

```
render() {
  const {
    title,
    navIcon,
  } = this.props;

  return (
    <header
      className={this.classes}
      style={this.getMergedStyles()}
      ref={this.topAppBarElement}
    >
      <div className='mdc-top-app-bar__row'>
        <section className='mdc-top-app-bar__section mdc-top-app-bar__section--align-start'>
          {navIcon ? navIcon : null}
          <span className="mdc-top-app-bar__title">
            {title}
          </span>
        </section>
        {this.renderActionItems()}
      </div>
    </header>
  );
}
```

這個 HTML 中有兩個 `section` 元素。第一個包含導覽圖示和標題。第二個則包含動作圖示。

接著，新增 `renderActionItems` 方法：

```
renderActionItems() {
  const {actionItems} = this.props;
  if (!actionItems) {
    return;
  }

  return (
    <section className='mdc-top-app-bar__section mdc-top-app-bar__section--align-end'
      role='toolbar'>
      {/* need to clone element to set key */}
      {actionItems.map((item, key) => React.cloneElement(item, {key}))}
    </section>
  );
}
```

開發人員會將 `TopAppBar` 匯入 React 應用程式，並將動作圖示傳遞至 `TopAppBar` 元素。您可以在 `App.js` 中查看初始化 `TopAppBar` 的範例程式碼。

缺少 `getMergedStyles` 方法，而 `render` 方法會使用該方法。請將下列 JavaScript 方法新增至 `TopAppBar` 類別：

```
getMergedStyles = () => {
  const {style} = this.props;
  const {style: internalStyle} = this.state;
  return Object.assign({}, internalStyle, style);
}
```

`render` 方法也缺少 `this.classes`，但我們會在後續章節中介紹。除了缺少 getter 方法 `this.classes` 之外，您仍需實作 `TopAppBar` 的部分，頂端應用程式列才能正確算繪。

頂端應用程式列仍缺少下列 React 元件：

- 已初始化的基礎
- 要傳遞至基礎架構的轉接介面方法
- JSX 標記
- 變體管理 (固定、簡短、一律摺疊、顯眼)

方法

1. 實作 *Adapter* 方法。
 2. 在 `componentDidMount` 中初始化 *Foundation*。
 3. 在 `componentWillUnmount` 中呼叫 *Foundation.destroy* 方法。
 4. 透過結合適當類別名稱的 getter 方法，建立變體管理機制。
-

步驟 4 · 約 4 分鐘

4. 實作轉接程式方法

非架構 JS `AppBar` `Component` 會實作下列 *Adapter* 方法 (詳情請參閱[這裡](#))：

- `hasClass()`
- `addClass()`
- `removeClass()`
- `registerNavigationIconInteractionHandler()`
- `deregisterNavigationIconInteractionHandler()`
- `notifyNavigationIconClicked()`
- `setStyle()`
- `getAppBarHeight()`
- `registerScrollHandler()`
- `deregisterScrollHandler()`
- `registerResizeHandler()`
- `deregisterResizeHandler()`
- `getViewportScrollY()`
- `getTotalActionItems()`

由於 React 有合成事件，以及不同的最佳編碼做法和模式，因此需要重新實作 *Adapter* 方法。

轉接器 Getter 方法

在 `AppBar.js` 檔案中，將下列 JavaScript 方法新增至 `AppBar`：

```
get adapter() {
  const {actionItems} = this.props;

  return {
    hasClass: (className) => this.classes.split(' ').includes(className),
    addClass: (className) => this.setState({classList: this.state.classList.add(className)}),
    removeClass: (className) => {
      const {classList} = this.state;
      classList.delete(className);
      this.setState({classList});
    },
    setStyle: this.setStyle,
    getAppBarHeight: () => this.topAppBarElement.current.clientHeight,
    registerScrollHandler: (handler) => window.addEventListener('scroll', handler),
    deregisterScrollHandler: (handler) => window.removeEventListener('scroll', handler),
    registerResizeHandler: (handler) => window.addEventListener('resize', handler),
    deregisterResizeHandler: (handler) => window.removeEventListener('resize', handler),
    getViewportScrollY: () => window.pageYOffset,
    getTotalActionItems: () => actionItems && actionItems.length,
  };
}
```

捲動和調整大小事件註冊的轉接器 API 實作方式與非架構 JS 版本相同，因為 React 沒有任何捲動或調整大小的[合成事件](#)，而是延遲至原生 DOM 事件系統。`getViewPortScrolly` 也是 `window` 物件上的函式，但 React 的 API 中沒有這個物件，因此也需要延遲至原生 DOM。每個架構的轉接程式實作方式都不相同。

有 3 種方法未實作：

- `registerNavigationIconInteractionHandler()`
- `deregisterNavigationIconInteractionHandler()`
- `notifyNavigationIconClicked()`

這是刻意設計，因為這 3 種方法是圍繞 DOM 事件設計，而這在 React 中是不良做法。請改用合成事件系統，並將事件監聽器附加至圖示元素。

您可能會發現缺少 `this.setStyle`，這是由 `get adapter` 方法呼叫。在 `TopAppBar.js` 檔案中，將缺少的 JavaScript 方法新增至 `TopAppBar` 類別：

```
setStyle = (varName, value) => {  
  const updatedStyle = Object.assign({}, this.state.style);  
  updatedStyle[varName] = value;  
  this.setState({style: updatedStyle});  
}
```

您剛實作了 *Adapter*！請注意，此時控制台可能會顯示錯誤，因為完整實作尚未完成。下一節將說明如何新增及移除 CSS 類別。

5. 實作元件方法

管理變體和類別

React 沒有管理類別的 API。如要模擬原生 JavaScript 的新增/移除 CSS 類別方法，請新增 `classList` 狀態變數。 `TopAppBar` 中有三段程式碼會與 CSS 類別互動：

1. `<TopAppBar />` 元件，方法是透過 `className` 屬性。
2. 透過 `addClass` 或 `removeClass` 的 *Adapter* 方法。
3. 在 `<TopAppBar />` React 元件中硬式編碼。

首先，請在 `TopAppBar.js` 頂端現有的匯入項目下方，新增下列匯入項目：

```
import classNames from 'classnames';
```

然後在 `TopAppBar` 元件的類別宣告中加入以下程式碼：

```
export default class TopAppBar extends React.Component {
  constructor(props) {
    super(props);
    this.topAppBarElement = React.createRef();
  }

  state = {
    classList: new Set(),
    style: {},
  };

  get classes() {
    const {classList} = this.state;
    const {
      alwaysCollapsed,
      className,
      short,
      fixed,
      prominent,
    } = this.props;

    return classNames('mdc-top-app-bar', Array.from(classList), className, {
      'mdc-top-app-bar--fixed': fixed,
      'mdc-top-app-bar--short': short,
      'mdc-top-app-bar--short-collapsed': alwaysCollapsed,
      'mdc-top-app-bar--prominent': prominent,
    });
  }

  ...
}
```

前往 <http://localhost:8080>，現在「Controls」核取方塊應該會切換 DOM 中的類別名稱。

這項程式碼可供許多開發人員使用 `TopAppBar`。開發人員可以與 `TopAppBar` API 互動，不必擔心 CSS 類別的實作細節。

注意：

這段程式碼新增了 3 項功能：

1. 頂端應用程式列 DOM 節點的參照。
2. 將 `classList` 狀態物件初始化為 `Set`。
3. 為計算類別字串的類別建立 `getter` 方法。

您現在已成功導入轉接程式。下一節將逐步引導您例項化 *Foundation*。

掛接及卸載元件

基礎例項會在 `componentDidMount` 方法中建立。

`TopAppBar` 實作 `componentDidMount` 方法，而非 `componentWillMount`，因為 *Foundation* 需要 DOM 元素。

首先，請在 `TopAppBar.js` 中現有的匯入項目後方加入下列匯入項目，匯入 MDC 頂端應用程式列基礎：

```
import {MDCTopAppBarFoundation, MDCFixedTopAppBarFoundation, MDCShortTopAppBarFoundation}
from '@material/top-app-bar';
```

接著，在 `TopAppBar` 類別中加入下列 JavaScript 程式碼：

```

export default class TopAppBar extends React.Component {

  ...

  foundation_ = null;

  componentDidMount() {
    this.initializeFoundation();
  }

  componentWillUnmount() {
    this.foundation_.destroy();
  }

  initializeFoundation = () => {
    if (this.props.short) {
      this.foundation_ = new MDCShortTopAppBarFoundation(this.adapter);
    } else if (this.props.fixed) {
      this.foundation_ = new MDCFxedTopAppBarFoundation(this.adapter);
    } else {
      this.foundation_ = new MDCTopAppBarFoundation(this.adapter);
    }

    this.foundation_.init();
  }

  ...

}

```

定義 propTypes 和 defaultProps 是良好的 React 程式碼設計做法。在 TopAppBar.js 中現有的匯入項目後方，新增下列匯入項目：

```
import PropTypes from 'prop-types';
```

接著在 TopAppBar.js 底部 (Component 類別之後) 新增下列程式碼：

```
import PropTypes from 'prop-types';

TopAppBar.propTypes = {
  alwaysCollapsed: PropTypes.bool,
  short: PropTypes.bool,
  fixed: PropTypes.bool,
  prominent: PropTypes.bool,
  title: PropTypes.string,
  actionItems: PropTypes.arrayOf(PropTypes.element),
  navIcon: PropTypes.element,
};

TopAppBar.defaultProps = {
  alwaysCollapsed: false,
  short: false,
  fixed: false,
  prominent: false,
  title: '',
  actionItems: null,
  navIcon: null,
};
```

您現在已成功實作 Top App Bar React 元件。前往 <http://localhost:8080> 即可使用示範頁面。這個示範網頁的運作方式與 [MDC Web 的示範網頁](#) 相同。示範頁面應如下所示：

6. 總結

在本教學課程中，我們說明如何包裝 MDC Web 的 *Foundation*，以便在 React 應用程式中使用。如「[整合至架構](#)」一文所述，Github 和 npm 上有幾個程式庫會包裝 MDC 網頁元件。建議您使用[這份清單](#)。除了 React 之外，這份清單也包含 Angular 和 Vue 等其他架構。

本教學課程將說明我們為何決定將 MDC Web 程式碼分成 3 個部分：*基礎*、*介面卡*和*元件*。這個架構可讓元件共用通用程式碼，同時與所有架構搭配運作。感謝您試用 Material Components React，請查看我們的新程式庫 [MDC React](#)。希望您喜歡這個程式碼研究室！
